Lecture 12 Nonparametric Regression I

STAT 441: Statistical Methods for Learning and Data Mining

Wenlu TANG

Department of Mathematical and Statistical Sciences
University of Alberta

Winter, 2025

Outline

1. Introduction
2. Polynomial Regression
3. Splines
4. Summary and Remark

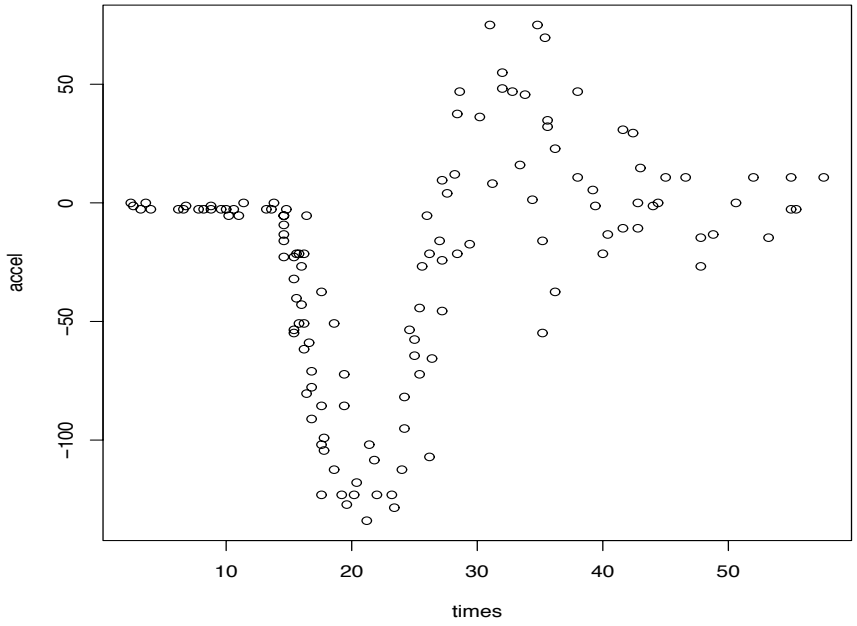
UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

Moving Beyond Linearity

- The truth is never linear! Or almost never!
- But often the linearity assumption is good enough.
- When its not ...
  - polynomials,
  - step functions,
  - splines,
  - local regression, and
  - generalized additive models
- offer a lot of flexibility, without losing the ease and interpretability of linear models.
- We observe $y_i = f(\mathbf{x}_i) + \varepsilon_i$ but no knowledge of $f(\cdot)$; determine $\hat{f}(\mathbf{x})$ from the data alone - no model.

Motorcycle Accident Dataset

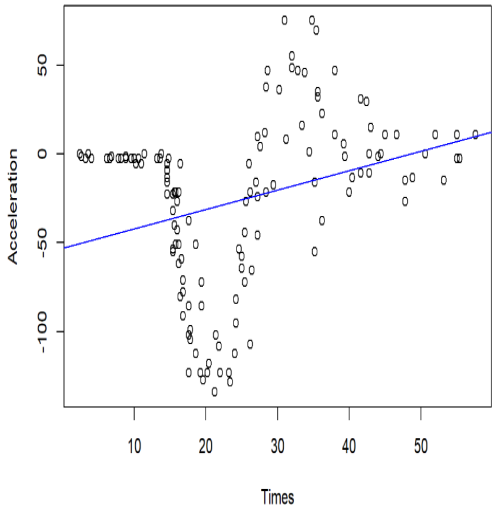
- Mcycle is a data frame giving a series of measurements of head acceleration in a simulated motorcycle accident, used to test crash helmets. It is an univariate case. We are interested in the relationship between the times in milliseconds after impact and the acceleration.




```
> import pandas as pd
> data=pd_csv('mcycle.csv', header=True)
> Y = mcycle['accel']
> X = mcycle['times']; plot(Y X, xlab="time",ylab="Accel")
```

Motorcycle Accident Dataset

- We can simply assume a linear relationship and run a linear model between acceleration and time.
- But after we draw the fitted linear line on the scatterplot, we can clearly tell that the linear assumption seems violated.




```
> lm-mod <- lm(Y X, data= mcycle); plot(X, Y, xlab="Times", ylab="Acceleration")
> abline(lm-mod, col="blue", lwd = 1)
```

Polynomial Regression

- If we assume there could be a polynomial relationship, we can try polynomial regression.
- One might try to fit a linear combination of certain basis functions, say orthogonal polynomials (see `help(poly)`)

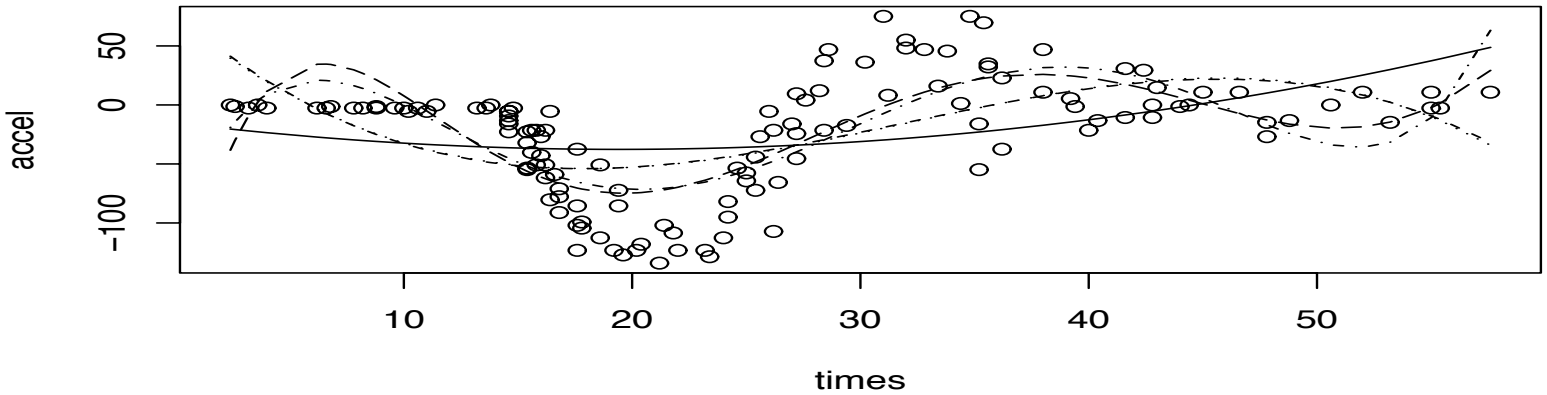
$$y_i = \beta_0 + x_i\beta_1 + \cdots + x_i^p\beta_p + \epsilon_i,$$

  essentially, a multiple linear regression.
- Not really interested in the coefficients; more interested in the fitted function values at any values x_0

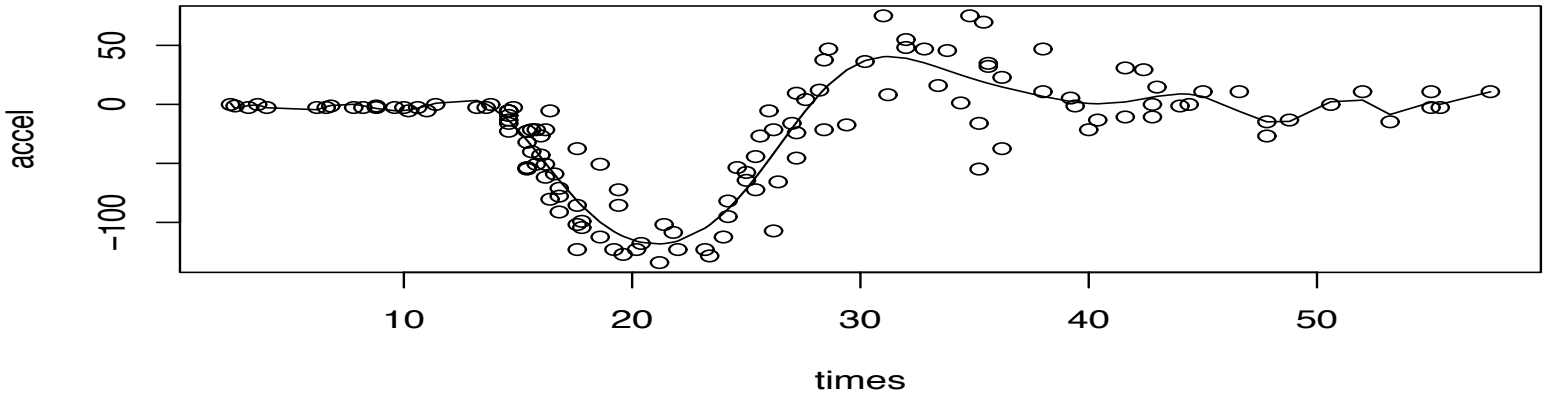
$$\hat{f}(x_0) = \beta_0 + x_0\beta_1 + \cdots + x_0^p\beta_p.$$

- Since $\hat{f}(x_0)$ is a linear function of $\hat{\beta}_j$, its variance can be easily obtained for pointwise-variances $\mathrm{Var}[\hat{f}(x_0)]$. Therefore the confidence interval of the prediction can be calculated.
- Polynomials can be very unstable to fit, and behave erratically away from the region where there are data.

Motorcycle Data



Polynomial fits of degrees 2,...,6 to motorcycle data



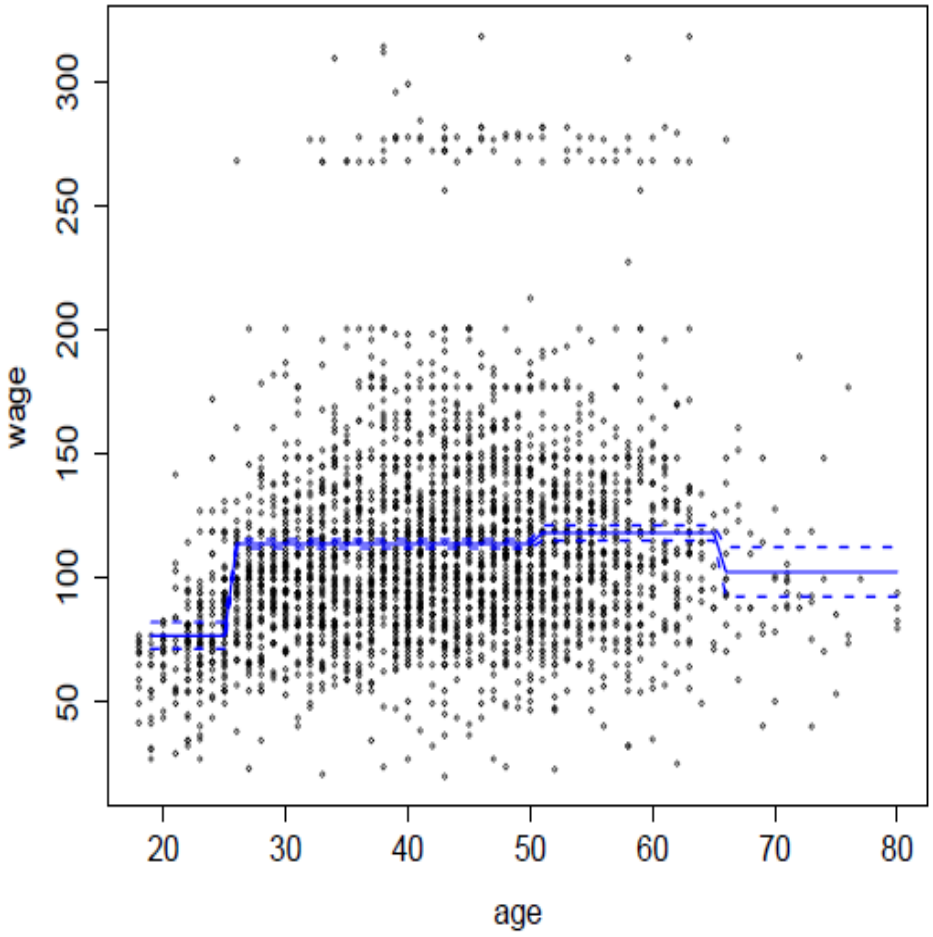
Polynomial fit of degree 20 (!) to motorcycle data

Step Functions

- Another way of creating transformations of a variable | cut the variable into distinct regions.

$$C_1(X) = I(X < \xi_1)\ C_2(X) = I(\xi_1 \leq X < \xi_2), \ldots, C_m(X \geq \xi_{m-1})$$

Results



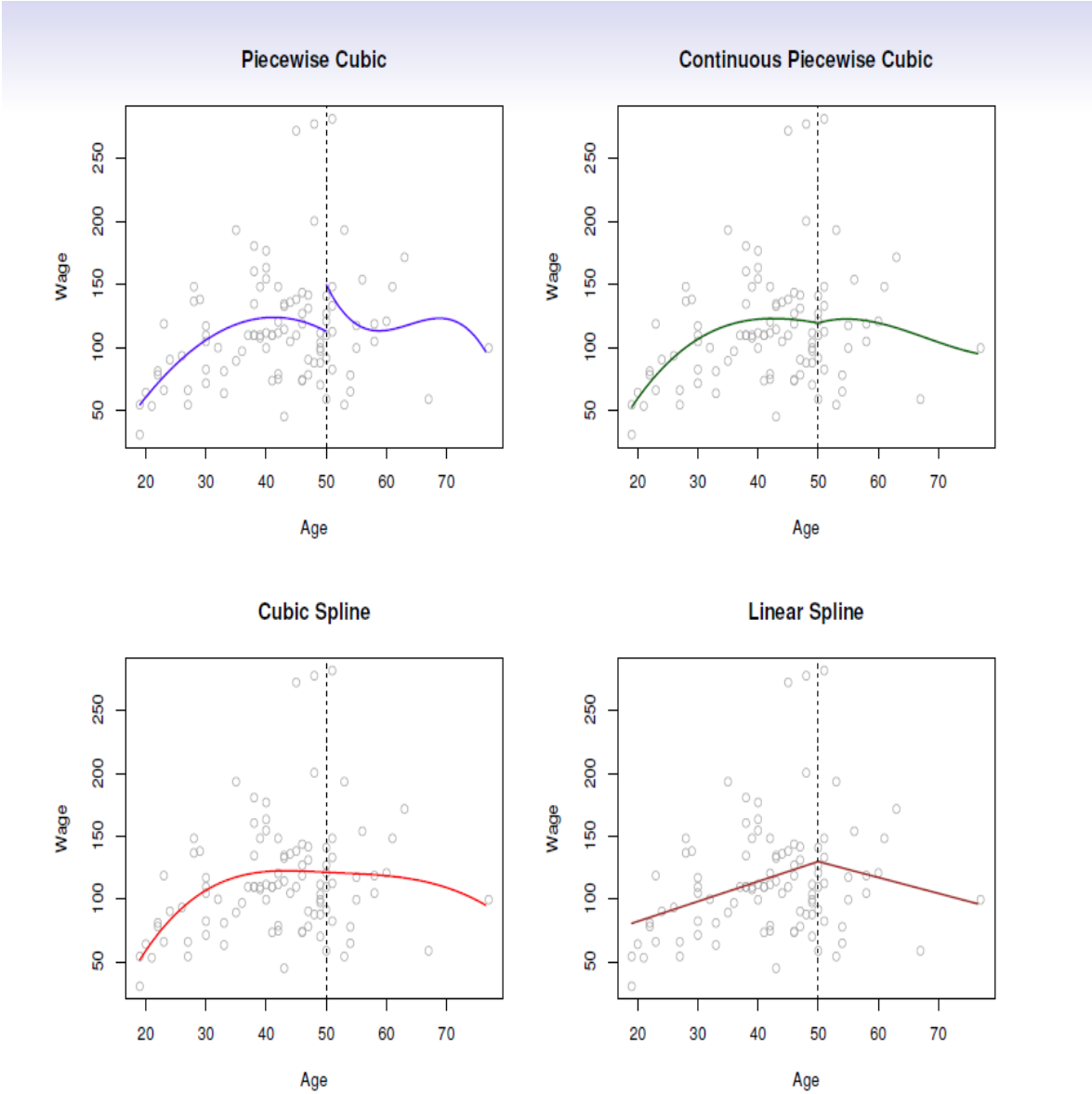
UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

Piecewise Polynomials

- Instead of a single polynomial in X over its whole domain, we can rather use different polynomials in regions defined by knots. E.g. (see figure)

$$y_i = \begin{cases} \beta_{01} + \beta_{11}x_i + \beta_{21}x_i^2 + \beta_{31}x_i^3 + \epsilon_i & \text{if } x_i < c \\ \beta_{02} + \beta_{12}x_i + \beta_{22}x_i^2 + \beta_{32}x_i^3 + \epsilon_i & \text{if } x_i \geq c \end{cases}$$

- Better to add constraints to the polynomials, e.g. continuity.
- Splines have the "maximum" amount of continuity.


Piecewise Cubic
Continuous Piecewise Cubic
Cubic Spline
Linear Spline
Wage
Age
50
100
150
200
250
20
30
40
50
60
70

Linear Splines

- A linear spline with knots at ε_k, $k = 1, \cdots, K$ is a piecewise linear polynomial continuous at each knot.
- We can represent this model as

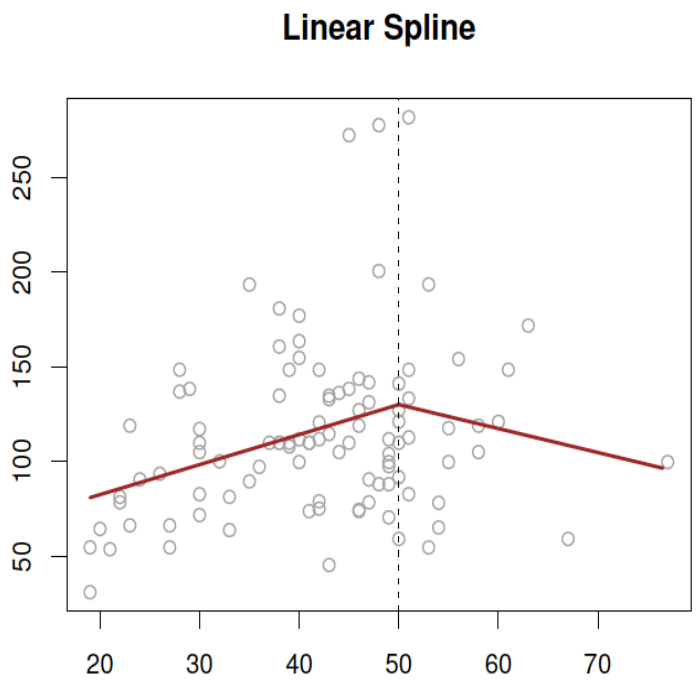
$$y_i = \beta_0 + \beta_1 b_1(x_i) + \cdots + \beta_{K+1} b_{K+1}(x_i) + \epsilon_i,$$

where the b_k are basis functions

$$b_1(x_i) = x_i,\ b_{k+1}(x_i) = (x_i - \varepsilon_i)_+,\ k = 1, \cdots, K,$$

where the $()_+$ means positive part, i.e.

$$(x_i - \varepsilon_i)_+ = \max(0, x_i - \varepsilon_i).$$



Cubic Splines

- A cubic spline with knots at ε_k, $k = 1, \cdots, K$ is a piecewise cubic polynomial with continuous derivatives up to order 2 at each knot.
- We can represent this model as

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \cdots + \beta_{K+3} b_{K+3}(x_i) + \epsilon_i,$$

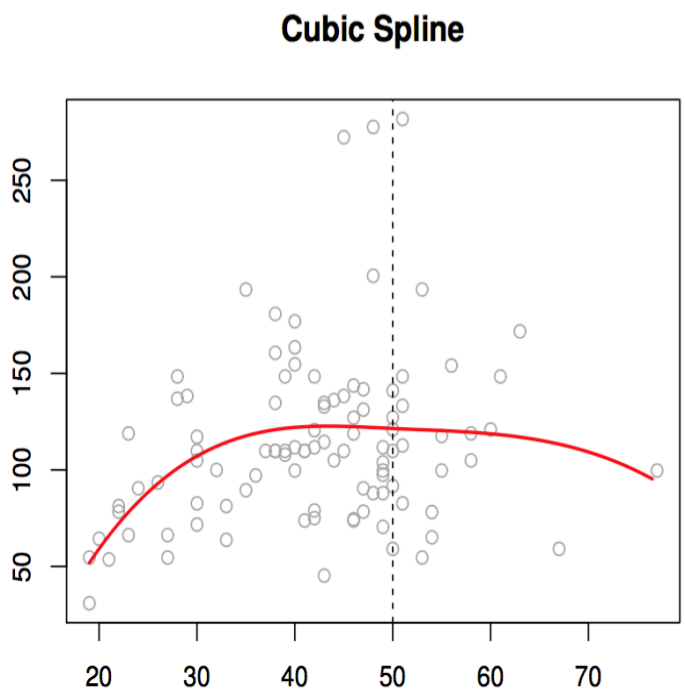
where the b_k are basis functions

$$b_1(x_i) = x_i,\ b_2(x_i) = x_i^2,\ b_3(x_i) = x_i^3,$$

$$b_{k+3}(x_i) = (x_i - \varepsilon_i)_+^3,\ k = 1, \cdots, K,$$

where the $()_+^3$ means positive part, i.e.

$$(x_i - \varepsilon_i)_+^3 = \max(0, (x_i - \varepsilon_i)^3).$$



UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

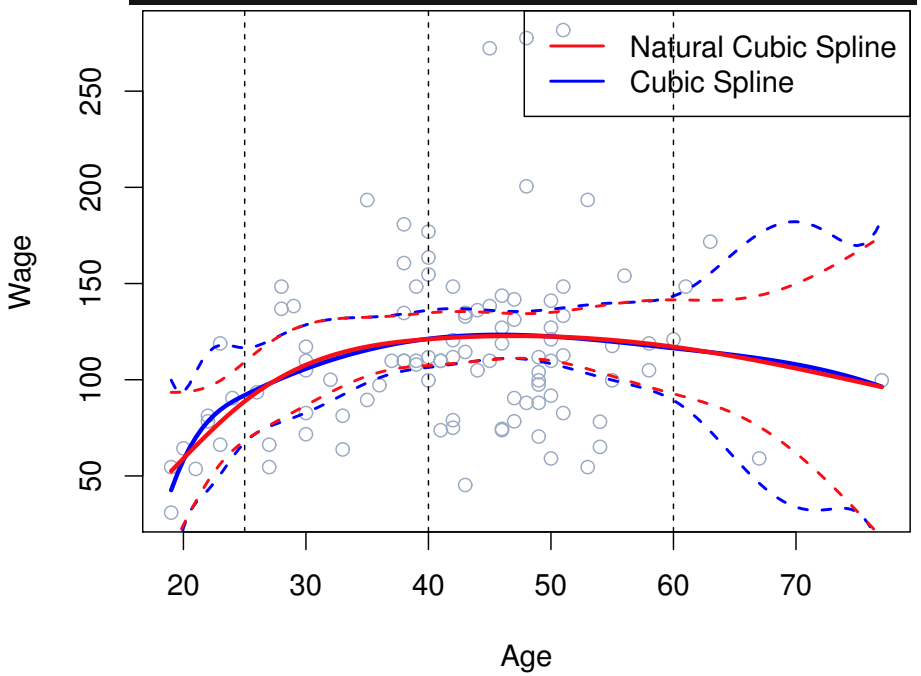
Natural Cubic Splines

2. Interpreting the Plot

Look at the **Wage vs. Age** plot on the right. Notice the dashed lines (confidence intervals):

- **The Purple Line (Regular Cubic Spline):** Notice how the dashed lines "flare out" significantly at the ends (near age 20 and age 80). This indicates high uncertainty; the model is "guessing" wildly because of the cubic flexibility.
- **The Red Line (Natural Cubic Spline):** The dashed lines are much tighter at the boundaries. Because it is forced to be a straight line at the ends, it is "constrained" and therefore more reliable.

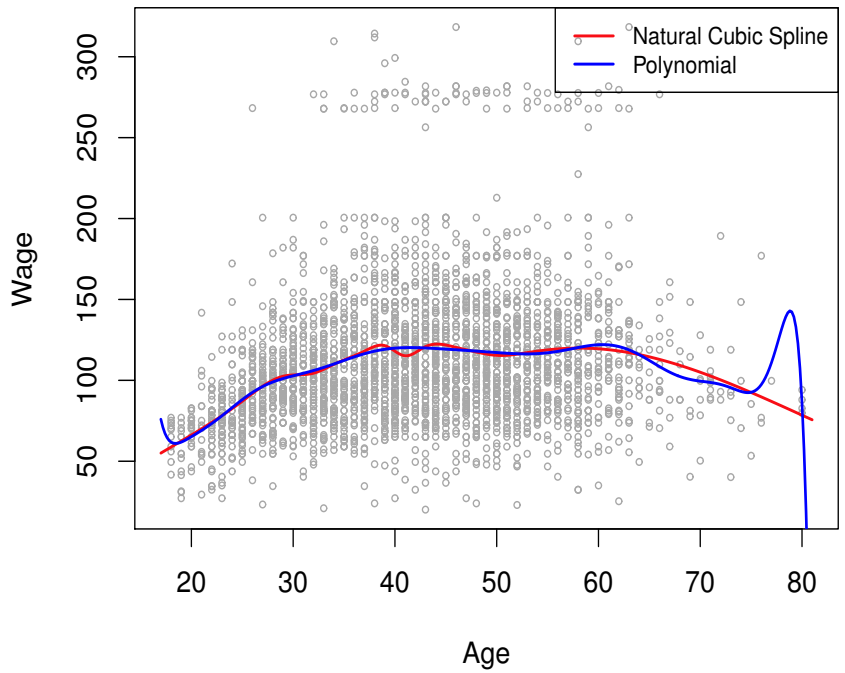
- A natural cubic spline extrapolates linearly beyond the boundary knots.
- This adds extra constraints, and allows us to put more internal knots for the same degrees of freedom as a regular cubic spline.



- Fitting splines in R is easy: bs(x, ...) for any degree splines, and ns(x, ...) for natural cubic splines, in package splines.
- Fitting splines in Python follows package: scipy.interpolate for interpolation of different basis, and use and b = BSpline(t, c, k) and b(x) for setting spline parameters.

Knot Placement

- One strategy is to decide K, the number of knots, and then place them at appropriate quantiles of the observed X.
- A cubic spline with K knots has $K + 4$ parameters or degrees of freedom.
- A natural spline with K knots has K degrees of freedom.



UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

Smoothing Splines

- Suppose now that we have data and to fit a smooth function $y = g(x)$ and we can achieve so by solving the penalized regression problem

$$\min_{g \in \mathcal{S}} \left\{ \sum_{i=1}^{N} \left(y_i - g\left(x_i\right)\right)^2 + \lambda \int_{x_1}^{x_N} \left[g''(x)\right]^2 dx \right\}, \tag{3.1}$$

  for a *smoothing parameter* $\lambda > 0$, where $\mathcal{S}$ is certain function space and can be $C^2\left[x_1, x_N\right]$.
- The first term is a fidelity term (RSS) and tries to make $g(x)$ match the data at each x.
- The smoothing parameter λ is a roughness penalty and controls how wiggly $g(x)$ is. It is modulated by the tuning parameter $\lambda \geq 0$.
- The smaller the λ, the more wiggly the function, eventually interpolating y_i when $\lambda = 0$.
- As $\lambda = \infty$, the function $g(x)$ becomes linear.

UNIVERSITY OF ALBERTA
EDMONTON·ALBERTA·CANADA

Smoothing Splines

- The roughness penalty term still controls the roughness via λ.
- Once the basis functions are determined, we have a parametric problem:

$$\min_{\theta} \left\{ \|\mathbf{y} - \mathbf{L}\theta\|^2 + \lambda \theta^T \mathbf{G} \theta \right\}$$

where $\mathbf{L}$: $n \times N$ has rows $\mathbf{b}^T(x_i)$ and $\boldsymbol{G}_{jk} = \int_{x_1}^{x_N} b_j''(x)\, b_k''(x)\, dx$.

- This gives

$$\hat{\boldsymbol{\theta}} = \left[\mathbf{L}^T\mathbf{L} + \lambda\mathbf{G}\right]^{-1} \mathbf{L}^T\mathbf{y}, \;\; \hat{\boldsymbol{y}} = L\hat{\boldsymbol{\theta}} = \mathbf{S}_\lambda \mathbf{y},$$

where the smoother matrix $\mathbf{S}_\lambda$ plays the same role as the hat matrix.

- The *equivalent degrees of freedom* are (thusly)

$$df_\lambda = tr\left[\mathbf{S}_\lambda\right].$$

Smoothing Splines continued

Some details

- Smoothing splines avoid the knot-selection issue, leaving a single λ to be chosen.
- The algorithmic details are too complex to describe here. In R, the function smooth.spline() will fit a smoothing spline.

Smoothing Splines

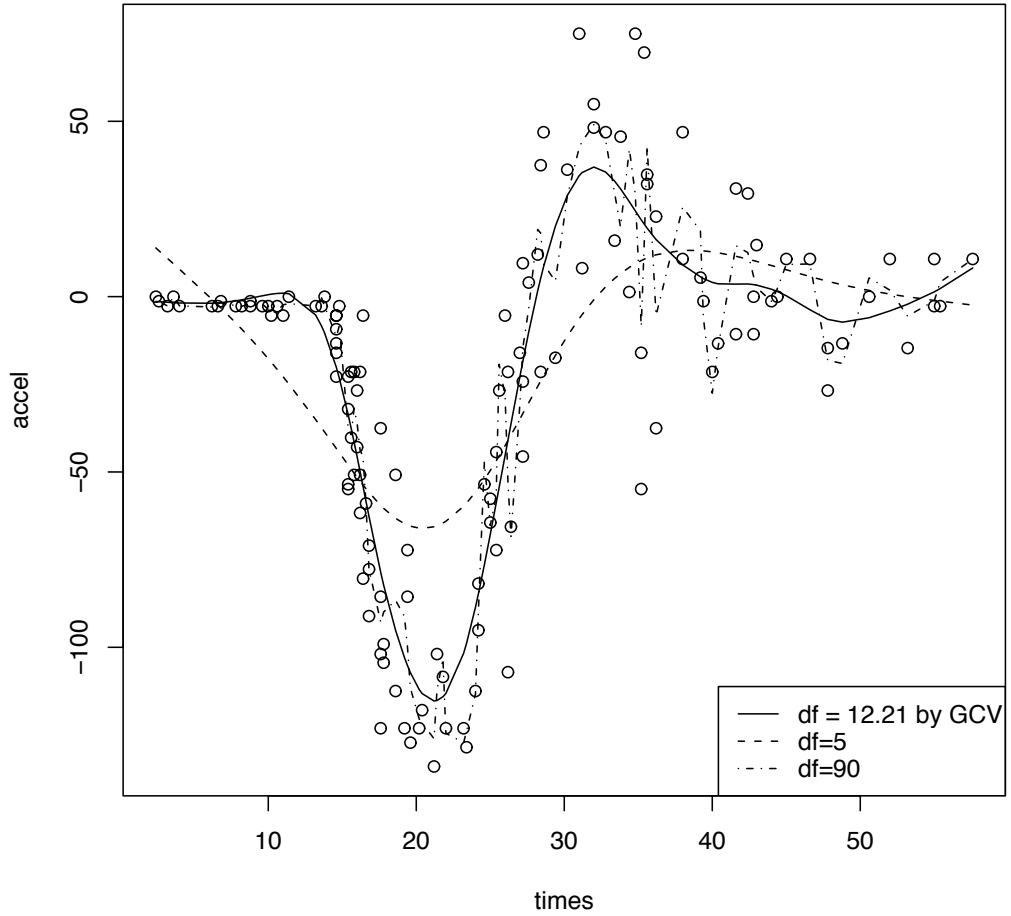
- The smoothness is controlled by λ; it and df_λ can be determined from each other.
- The R function will determine an optimal λ_0 by generalized cross-validation:

$$\lambda_0 = \arg\min_\lambda \frac{\|(\mathbf{I} - \mathbf{S}_\lambda)\,\mathbf{y}\|^2}{n - df_\lambda}.$$

- This is the default; an option is ordinary cross-validation:

$$\lambda_0 = \arg\min_\lambda \sum_{i=1}^{n} \left(\frac{e_i(\lambda)}{1 - [\mathbf{S}_\lambda]_{ii}} \right)^2.$$

Motorcycle Data



Summary and Remark

- Polynomial regression
- Splines
- Read textbook Chapter 7 and code